

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Timus
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

1296. Гиперпереход	3
Условие:	3
Решение:.....	4
О решении:	5
2025. Стенка на стенку	6
Условие:	6
Решение:.....	7
О решении:	8

1296. Гиперпереход

Условие:

Гиперпереход, открытый ещё в начале XXI-го века, и сейчас остаётся основным способом перемещения на расстояния до сотен тысяч парсеков. Но совсем недавно физиками открыто новое явление. Оказывается, длительностью альфа-фазы перехода можно легко управлять. Корабль, находящийся в альфа-фазе перехода, накапливает гравитационный потенциал. Чем больше накопленный гравитационный потенциал корабля, тем меньше энергии потребуется ему на прыжок сквозь пространство. Ваша цель — написать программу, которая позволит кораблю за счёт выбора времени начала альфа-фазы и её длительности накопить максимальный гравитационный потенциал.

В самой грубой модели грави-интенсивность — это последовательность целых чисел p_i . Будем считать, что если альфа-фаза началась в момент i и закончилась в момент j , то накопленный в течение альфа-фазы потенциал — это сумма всех чисел, стоящих в последовательности на местах от i до j .

Исходные данные

В первой строке записано целое число N — длина последовательности, отвечающей за грави-интенсивность ($0 \leq N \leq 60000$). Далее идут N строк, в каждой записано целое число p_i ($-30000 \leq p_i \leq 30000$).

Результат

Выведите максимальный гравитационный потенциал, который может накопить корабль в альфа-фазе прыжка. Считается, что потенциал корабля в начальный момент времени равен нулю.

Примеры

исходные данные	результат
10 31 -41 59 26 -53 58 97 -93 -23 84	187
3 -1 -5 -6	0

Решение:

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cin >> n ;

    long long curr = 0, best = 0;
    for (int i = 0; i < n; i++) {
        int curr_val;
        cin >> curr_val;

        curr = max(0LL, curr + curr_val);
        best = max(best, curr);
    }

    cout << best << endl;
}
```

О решении:

Идея:

Идем по массиву и прибавляем к текущей максимальной сумме подмассива (в начале 0) попавшийся элемент, выбираем лучший вариант – оставляем (если текущая сумма получилась отрицательной, то лучше начать заново с нуля). Таким образом на каждой итерации у нас будет лучшая сумма подмассива с концом в индексе i . И на каждой итерации обновляем общий лучший максимум.

Сложность по времени:

$O(n)$ – так как линейно проходимся по числам.

Сложность по памяти:

$O(1)$ – так как не храним массив, а сумму обрабатываем итеративно, без хранения массива.

2025. Стенка на стенку

Условие:

Бокс, каратэ, самбо... Классические боевые единоборства пресытили аудиторию. Поэтому известный спортивный канал запускает новый формат соревнований, основанный на традиционной русской забаве — боях стенка на стенку. В соревновании могут участвовать от двух до k команд, каждая из которых будет соперничать с остальными. Всего в соревновании примут участие n бойцов. Перед началом боя они должны разделиться на команды, каждый боец должен войти ровно в одну команду. За время боя два бойца сразятся, если они состоят в разных командах. Организаторы считают, что популярность соревнований будет тем выше, чем больше будет количество схваток между бойцами. Помогите распределить бойцов по командам так, чтобы максимизировать количество схваток между бойцами, и выведите это количество.

Исходные данные

В первой строке дано количество тестов T ($1 \leq T \leq 10$). В следующих T строках перечислены тесты. В каждой из них записаны целые числа n и k через пробел ($2 \leq k \leq n \leq 10^4$).

Результат

Для каждого теста в отдельной строке выведите одно целое число — ответ на задачу.

Пример

исходные данные	результат
3	12
6 3	10
5 5	4
4 2	

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

long long get_max_fights(int fighters, int teams) {
    int fighters_per_team = fighters / teams;
    int extra_fighters = fighters - fighters_per_team * teams;
    long long total_fights = 0;

    vector<int> team_sizes(teams, fighters_per_team);
    for (int i = 0; i < extra_fighters; i++) {
        team_sizes[i]++;
    }

    for (int i = 0; i < (int)team_sizes.size(); i++) {
        total_fights += team_sizes[i] * (fighters - team_sizes[i]);
    }

    return total_fights / 2;
}

int main() {
    int T;
    cin >> T;

    for (int i = 0; i < T; i++) {
        int fighters, teams;
        cin >> fighters >> teams;

        cout << get_max_fights(fighters, teams) << endl;
    }
}
```

О решении:

Идея:

Максимальное число боев будет, когда у нас в командах равное число участников (или хотя бы максимум разница в единицу), так как чем меньше пар внутри одной команды, тем больше пар между разными командами. Тогда, распределив максимально поровну участников по командам, просуммируем значения боев для каждой команды (каждый человек в команде i сразится со всеми остальными из других команд). Также делим получившийся результат на два, так как каждую драку мы учли дважды.

Сложность по времени:

$O(k)$ – так как для каждой из k команд считаем количество их боев.

Сложность по памяти:

$O(k)$ – так как храним массив длиной в количество команд.